



Centro de
Informática
UFPE

Análise de Métricas e Interações de Features no BusyBox 1.38.0 utilizando o PCLocator

- Introdução e contextualização;
- Estudo de caso - Busybox 1.38.0;
- Metodologia e Automação;
- Métricas Coletadas e Definições;
- Resultados - Visão Geral (LoC Agregado);
- Complexidade e interações de Alto Grau;
- Desafios Técnicos - O Problema da Feature Opcional;
- Conclusões;

Introdução e Contextualização

Linha de Produtos de Software (LPS):

Uma linha de produtos é um conjunto de produtos do portfólio de um fabricante que compartilham semelhanças substanciais e que, idealmente, são criados a partir de um conjunto de peças reutilizáveis.

Features:

Unidades incrementais de funcionalidade que distinguem os produtos dentro de uma LPS

Objetivo do Trabalho:

Extrair e analisar métricas de variabilidade (LoC e interações) para quantificar a complexidade de um sistema altamente configurável.

Estudo de Caso BusyBox 1.38.0

BusyBox:

Conhecido como o "canivete suíço" do Linux embarcado, é um exemplo clássico de sistema que utiliza variabilidade baseada em anotações (diretivas de pré-processador C)

Escopo da Análise:

Foi analisada a versão 1.38.0, focando no mapeamento de features realizado pela ferramenta [PCLocator](#).

Metodologia e Automação

Processo de Coleta:

Devido à grande quantidade de arquivos .c, a coleta manual seria inviável.

Desenvolvimento de Script Python:

Foi criado um script para automatizar a varredura das pastas do BusyBox.

Fluxo de Dados:

O script executa o PCLocator para cada arquivo e redireciona a saída para arquivos .txt individuais, permitindo o processamento posterior dos dados.

Métricas Coletadas e Definições

Lines of Code (LoC):

Quantidade de linhas de código vinculadas a uma feature ou combinação delas.

Feature Interactions (n-way):

Ocorre quando o comportamento (ou a presença) de uma feature depende da seleção (ou não seleção) de outras.

Grau de Interação:

Refere-se à quantidade de features envolvidas em um bloco de código (ex: Grau 2 envolve a interação de duas features como A && B)

Resultados - Visão Geral (LoC Agregado)

Volume Total Analisado:

77.601 linhas de código mapeadas sob condições de variabilidade

Distribuição por Grau:

- *Grau 1 (Features isoladas): 53.578 LoC (69,04% do total).*
- *Grau 2 (Interações de 2-vias): 14.892 LoC (19,19%).*
- *Grau 3 (Interações de 3-vias): 4.927 LoC (6,35%)*

A maior parte do código configurável do BusyBox é simples (Grau 1), mas quase 20% do sistema reside em interações de Grau 2, indicando acoplamento significativo entre funcionalidades.

Distribuição por Grau

%LoC / Grau

4

2,5%

3

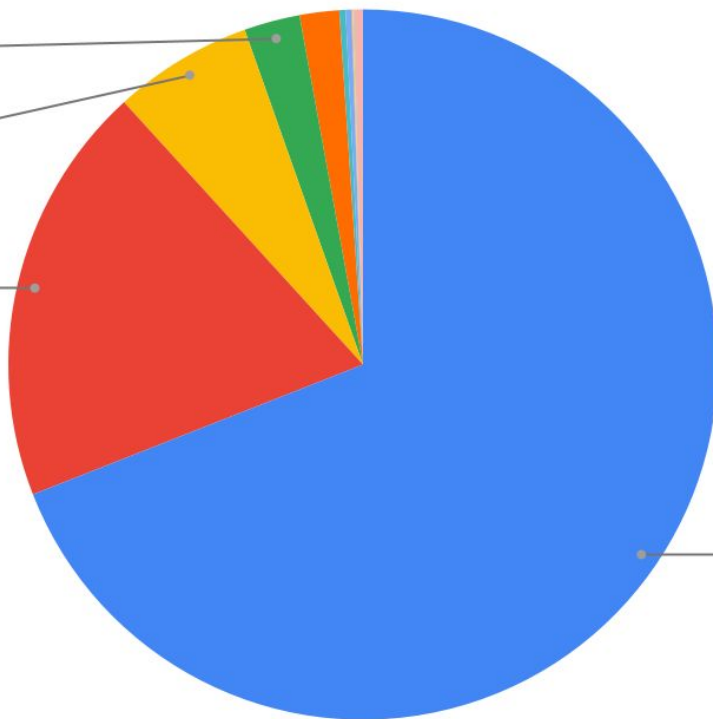
6,3%

2

19,2%

1

69,0%



Complexidade e Interações de Alto Grau (Higher-order)

Interações Complexas:

Foram detectadas interações de até Grau 19 (5 LoC), embora representem apenas 0,01% do sistema.

Exemplos Reais:

No arquivo addgroup.txt, uma interação de Grau 2:

*ENABLE_FEATURE_SHADOWPASSWDS &&
ENABLE_FEATURE_ADDUSER_TO_GROUP
controla a linha 193.*

No arquivo alg-yescript-common.txt, condições negativas e positivas combinadas: !RESTRICTED_PARAMS && TEST_DECODE64 geram 37 LoC de interação

Arquivo: addgroup.txt

```
192 | # if ENABLE_FEATURE_SHADOWPASSWDS | ENABLE_FEATURE_ADDUSER_TO_GROUP |
193 | update_passwd(bb_path_gshadow_file, ar | ENABLE_FEATURE_SHADOWPASSWDS&&ENABLE_FEATURE_ADDUSER_TO_GROUP |
194 | # endif | ENABLE_FEATURE_ADDUSER_TO_GROUP |
```

Arquivo: alg-yescrypt-common.txt

```
107 | #if TEST_DECODE64 | !RESTRICTED_PARAMS |
108 | static void test_decode64_uint32(void) | !RESTRICTED_PARAMS&&TEST_DECODE64 |
109 | { | !RESTRICTED_PARAMS&&TEST_DECODE64 |
110 |     const uint8_t *src, *end; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
111 |     uint32_t u32; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
112 |     int a = 48; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
113 |     int b = 8<<6; // 0x0200 | !RESTRICTED_PARAMS&&TEST_DECODE64 |
114 |     int c = 4<<12; // 0x04000 | !RESTRICTED_PARAMS&&TEST_DECODE64 |
115 |     int d = 2<<18; // 0x080000 | !RESTRICTED_PARAMS&&TEST_DECODE64 |
116 |     int e = 1<<24; // 0x1000000 | !RESTRICTED_PARAMS&&TEST_DECODE64 |
117 | | !RESTRICTED_PARAMS&&TEST_DECODE64 |
118 |     src = (void*)"wzzz"; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
119 |     end = decode64_uint32(&u32, src, 0); | !RESTRICTED_PARAMS&&TEST_DECODE64 |
120 |     if (u32 != 0x0003ffff+c+b+a) bb_error_m | !RESTRICTED_PARAMS&&TEST_DECODE64 |
121 |     if (end != src + 4) bb_error_msg_and_di | !RESTRICTED_PARAMS&&TEST_DECODE64 |
122 |     src = (void*)"xzzz"; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
123 |     end = decode64_uint32(&u32, src, 0); | !RESTRICTED_PARAMS&&TEST_DECODE64 |
124 |     if (u32 != 0x0007ffff+c+b+a) bb_error_m | !RESTRICTED_PARAMS&&TEST_DECODE64 |
125 |     if (end != src + 4) bb_error_msg_and_di | !RESTRICTED_PARAMS&&TEST_DECODE64 |
126 |     // Note how the last representable "x-- | !RESTRICTED_PARAMS&&TEST_DECODE64 |
127 |     // And if we now increment it, we get: | !RESTRICTED_PARAMS&&TEST_DECODE64 |
128 |     src = (void*)"y..."; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
129 |     end = decode64_uint32(&u32, src, 0); | !RESTRICTED_PARAMS&&TEST_DECODE64 |
130 |     if (u32 != 0x00000000+d+c+b+a) bb_error | !RESTRICTED_PARAMS&&TEST_DECODE64 |
131 |     if (end != src + 5) bb_error_msg_and_di | !RESTRICTED_PARAMS&&TEST_DECODE64 |
132 |     src = (void*)"yzzzz"; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
133 |     end = decode64_uint32(&u32, src, 0); | !RESTRICTED_PARAMS&&TEST_DECODE64 |
134 |     if (u32 != 0x00ffffff+d+c+b+a) bb_error | !RESTRICTED_PARAMS&&TEST_DECODE64 |
135 |     if (end != src + 5) bb_error_msg_and_di | !RESTRICTED_PARAMS&&TEST_DECODE64 |
136 | | !RESTRICTED_PARAMS&&TEST_DECODE64 |
137 |     src = (void*)"zzzzzz"; | !RESTRICTED_PARAMS&&TEST_DECODE64 |
138 |     end = decode64_uint32(&u32, src, 0); | !RESTRICTED_PARAMS&&TEST_DECODE64 |
139 |     if (u32 != 0x3fffffff+e+d+c+b+a) bb_err | !RESTRICTED_PARAMS&&TEST_DECODE64 |
140 |     if (end != src + 6) bb_error_msg_and_di | !RESTRICTED_PARAMS&&TEST_DECODE64 |
141 | | !RESTRICTED_PARAMS&&TEST_DECODE64 |
142 |     bb_error_msg("test_decode64_uint32() OK | !RESTRICTED_PARAMS&&TEST_DECODE64 |
143 | } | !RESTRICTED_PARAMS&&TEST_DECODE64 |
144 | #else | !RESTRICTED_PARAMS |
```

...

```
308 | #if TEST_DECODE64 | True |
309 | test_decode64_uint32(); | !RESTRICTED_PARAMS&&TEST_DECODE64 |
310 | | True |
```

Desafios Técnicos - O Problema da Feature Opcional

Definição:

Surge quando a interação entre duas features opcionais exige um código de coordenação que só deve estar presente se ambas forem selecionadas.

Relação com os Dados:

Os blocos de código com Grau ≥ 2 nos resultados do PCLocator representam exatamente esse "código de coordenação" necessário para gerenciar a coexistência de múltiplas features.

Conclusão

Eficácia da Automação:

O script integrado com o PCLocator permitiu uma análise sistêmica que revela a "Virtual Separation of Concerns" (separação virtual de preocupações) presente no BusyBox.

Impacto na Manutenção:

Interações de alto grau (como a de Grau 19 encontrada) representam riscos de manutenção, pois são difíceis de testar exaustivamente devido à explosão combinatória.



**Muito
obrigado!**



Centro de
Informática
UFPE